Identity Protocols 2026

Beyond the OAuth basics

Marc Müller / Principal Consultant · 4tecture

YOUR SPEAKER

Marc Müller

Principal Consultant · 4tecture
Technical Lead & Architect · ProAuth

marcromanmueller

marc-mueller

@muellermarc

ProAuth

Multi-tenant OpenID Connect identity provider

proauth.net

ReaFx

Enterprise application framework for .NET

reafx.net

RESOURCES

Take the slides with you

SLIDES · REFERENCES · FOLLOW-UP

www.4tecture.ch/events/codetalks26-identityprotocols

The One Rule

Stop designing for token possession.

Start designing for token context.

Why This Changed

- Browser apps grew heavier
- APIs call more APIs
- Agents act without humans
- Bearer leakage is routine
- Zero-trust needs narrower claims

RFC 9700 · RFC 6750 · Browser-based apps · RFC 10017

How Tokens Escape

01 **Browser runtime** ✓

Injected scripts, storage and debugging tools.

02 **Infrastructure** ✓

Reverse proxies, request traces and telemetry.

03 **Backend operations** ✓

Queues, retries and crash reports.

04 **Human workflows**

Developer machines, copied curl commands and support tickets.

RFC 6750 · RFC 9700

Follow One API Journey

01 / BASELINE

Issue safely

Bind the grant to the right transaction.

02 / SENDER

Resist replay

Require proof of the bound key.

03 / BROWSER

Reduce exposure

Choose where tokens can be read.

04 / DELEGATION

Limit authority

Give each service its approved access.

05 / ASSURANCE

Protect intent

Express and protect sensitive authorization.

The 2026 Baseline

Start with a secure baseline for interactive authorization.

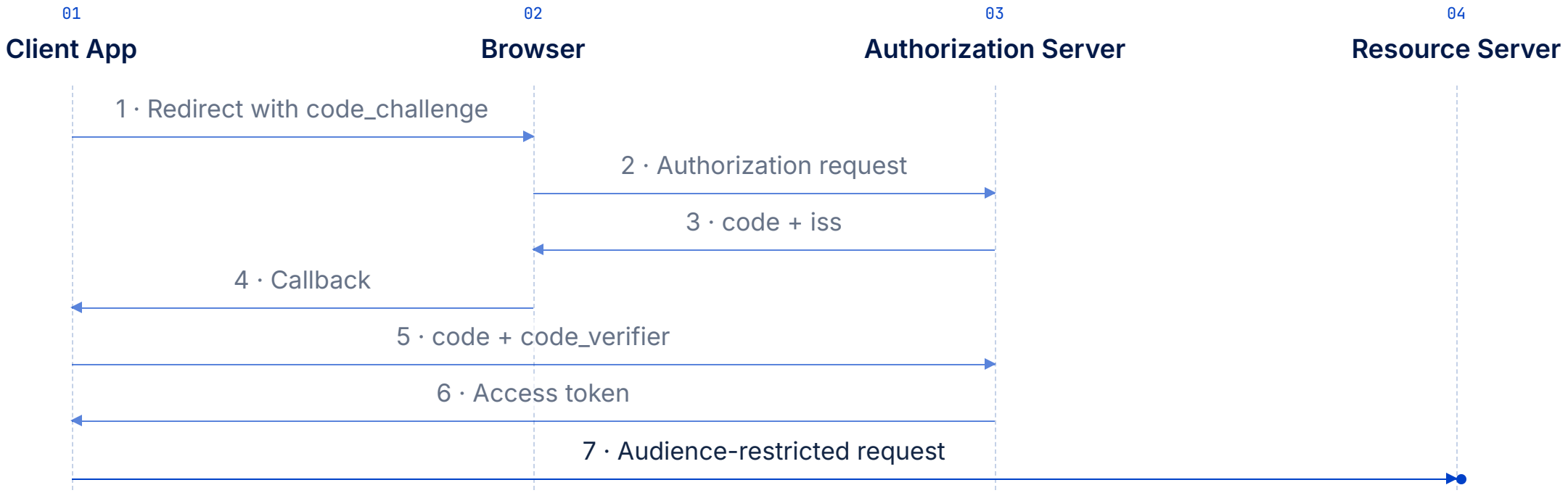
01

Recommended Posture

OLD HABIT / GAP	RECOMMENDED POSTURE
Implicit flow / collecting user passwords	Code + S256 PKCE; redirect to the AS
Loose redirect handling	Exact redirect matching
Multiple issuers without transaction binding	Expected issuer + response validation
Reusable public-client refresh credentials	Rotation or sender constraint
Long-lived shared client secrets	Prefer asymmetric client authentication where feasible

RFC 9700 · RFC 9207 · OAuth 2.1 draft

Baseline Flow

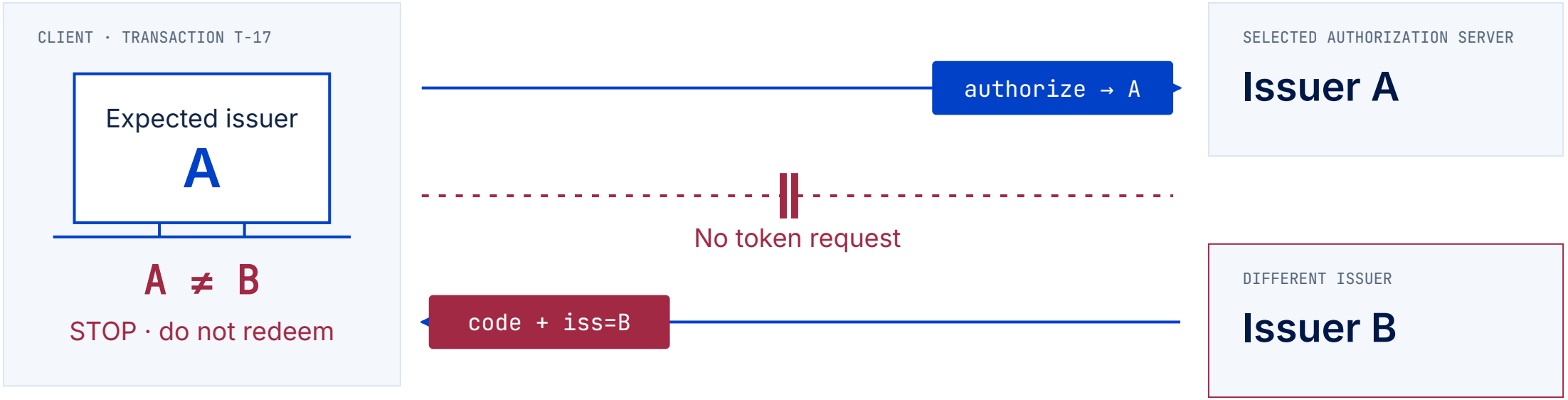


TRACE 7 / 7 **Audience-restricted request**

RFC 7636 · RFC 9207 · RFC 9700

Bind The Response To The Right Issuer

REMEMBER | AUTHORIZE | RECEIVE | COMPARE | STOP



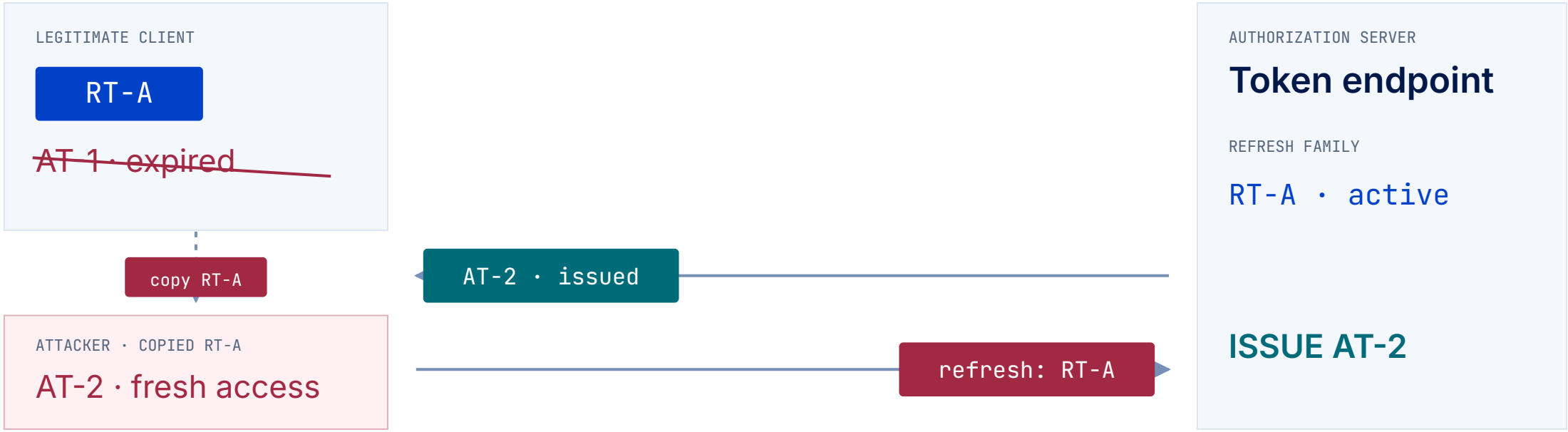
05 Mismatch: stop before sending the code to any token endpoint.

> Simplified issuer check. The callback must also be correlated with its initiating transaction.

RFC 9207 · RFC 9700

A Stolen Refresh Token Extends Access

ISSUE | COPY | EXPIRE | REDEEM | NEW ACCESS



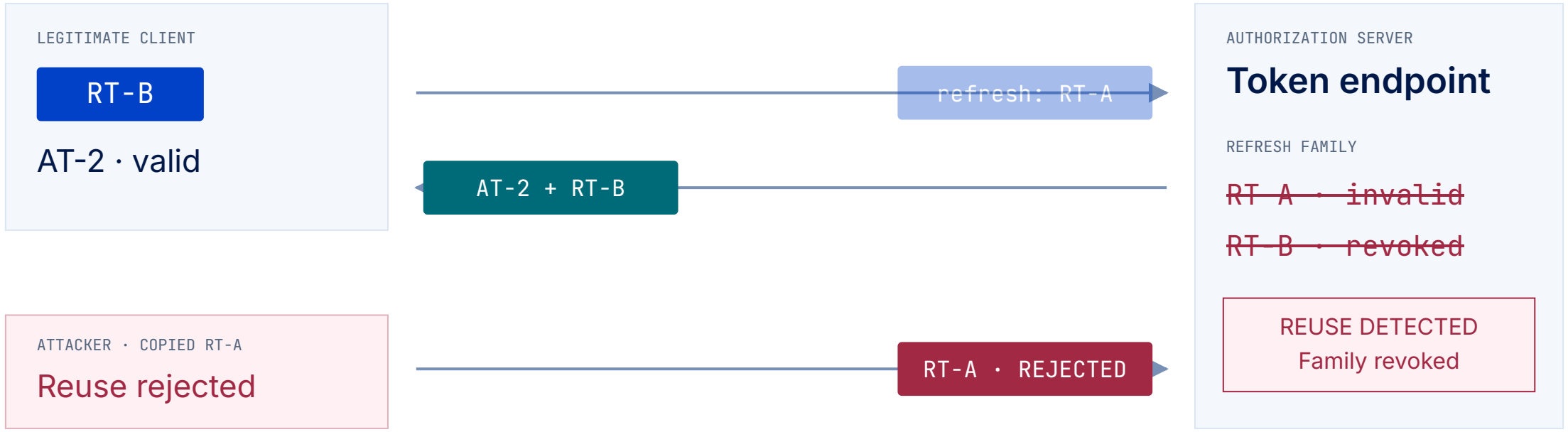
05 The AS issues AT-2: access continues beyond the lifetime of AT-1.

> Attack assumptions: public client; no rotation, no sender constraint; RT-A remains valid.

RFC 9700

Rotation Makes Refresh-Token Reuse Visible

STOLEN COPY | REDEEM | ROTATE | REUSE | DETECT | REVOKE



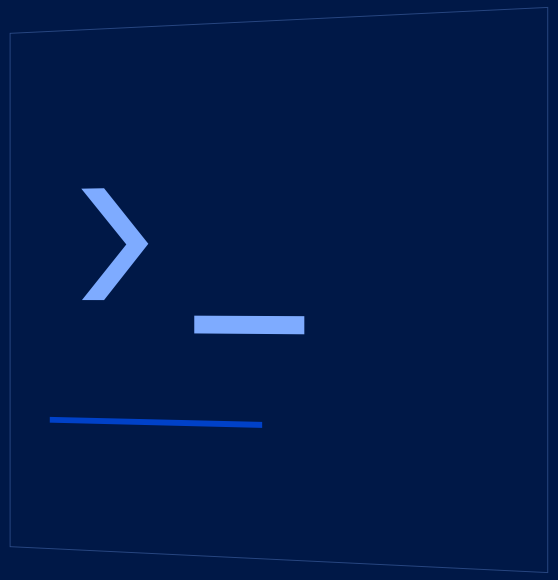
06 Reject reuse and revoke RT-B, the active token in this refresh family.

> Reuse revokes the refresh family. Already-issued access tokens may remain valid.

RFC 9700

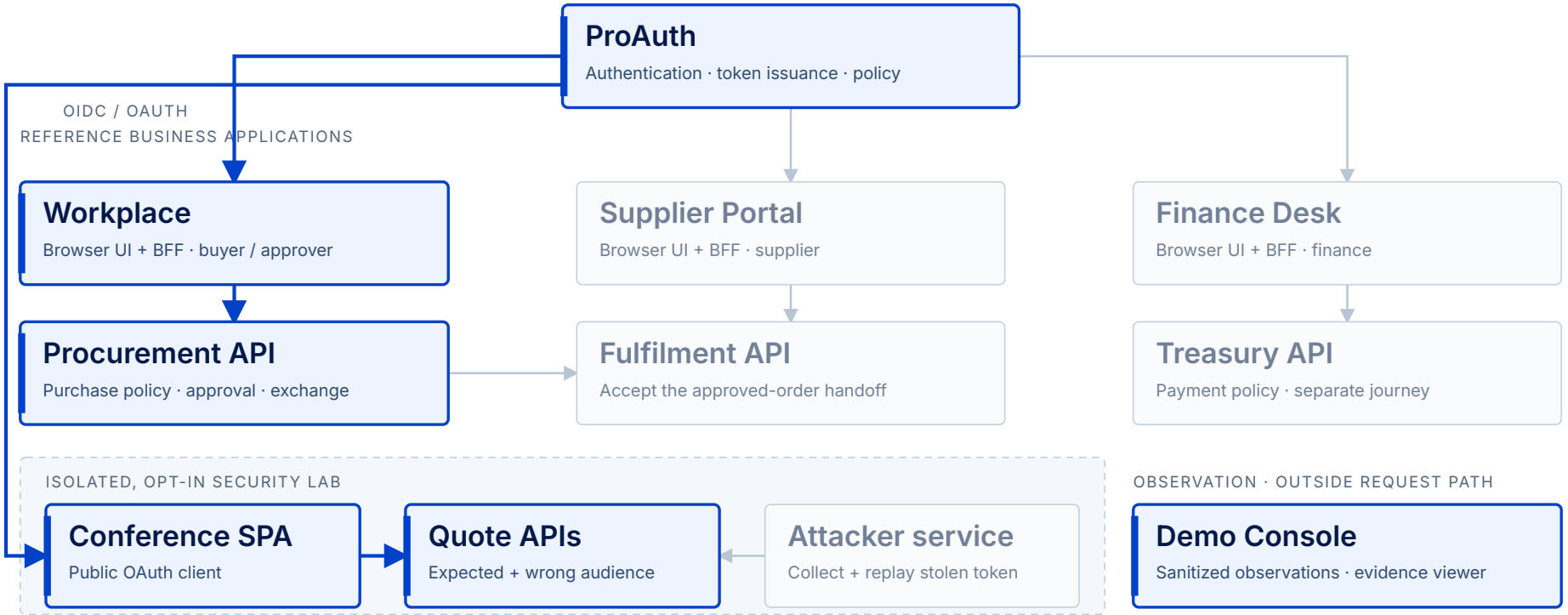
Security baseline

Follow the code. Check the issuer. Rotate the refresh token.



Two places to inspect the baseline

In this demo Grey components provide context



Workplace sign-in first. Then the isolated quote client tests audience and refresh boundaries.

Reference implementation: architecture.md · conference.md · api-assurance.md. Logical overview; infrastructure and optional workers omitted.

Mervikon Group

WORKPLACE

Procurement 01

Equipment catalogue 02

PURCHASING

Equipment for the work ahead.

Request equipment, review your purchases and approve within your authority.

Sign in to see your purchases. Access is checked by the Procurement API.

Your purchases

0 visible

EQUIPMENT	VALUE	STATUS

Equipment catalogue

CHF · demonstration fixtures

01 **Industrial Laser Calibration Station**
Standard approval · separation of duties applies.
CHF 18'500 Request

02 **Mission-Critical Coffee Infrastructure**
Approval requires verified MFA or phishing-resistant authentication within five minutes.
CHF 24'900 Request

03 **Autonomous Office Assistant Robot**
CHF 42'000 Request

Baseline: Secure Issuance And Renewal

RISK

Misissued or stolen access

A response reaches the wrong flow, or a stolen credential keeps renewing.

CONTROL

Bind and protect

S256 PKCE, exact redirects, issuer correlation and refresh replay protection.

STILL REQUIRED

API enforcement

Validate the API audience and enforce the requested operation.

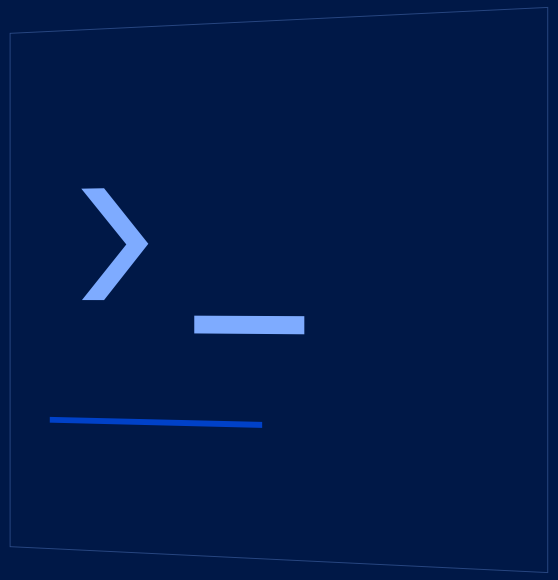
Sender-Constrained Tokens

02

A copied token should not be enough.

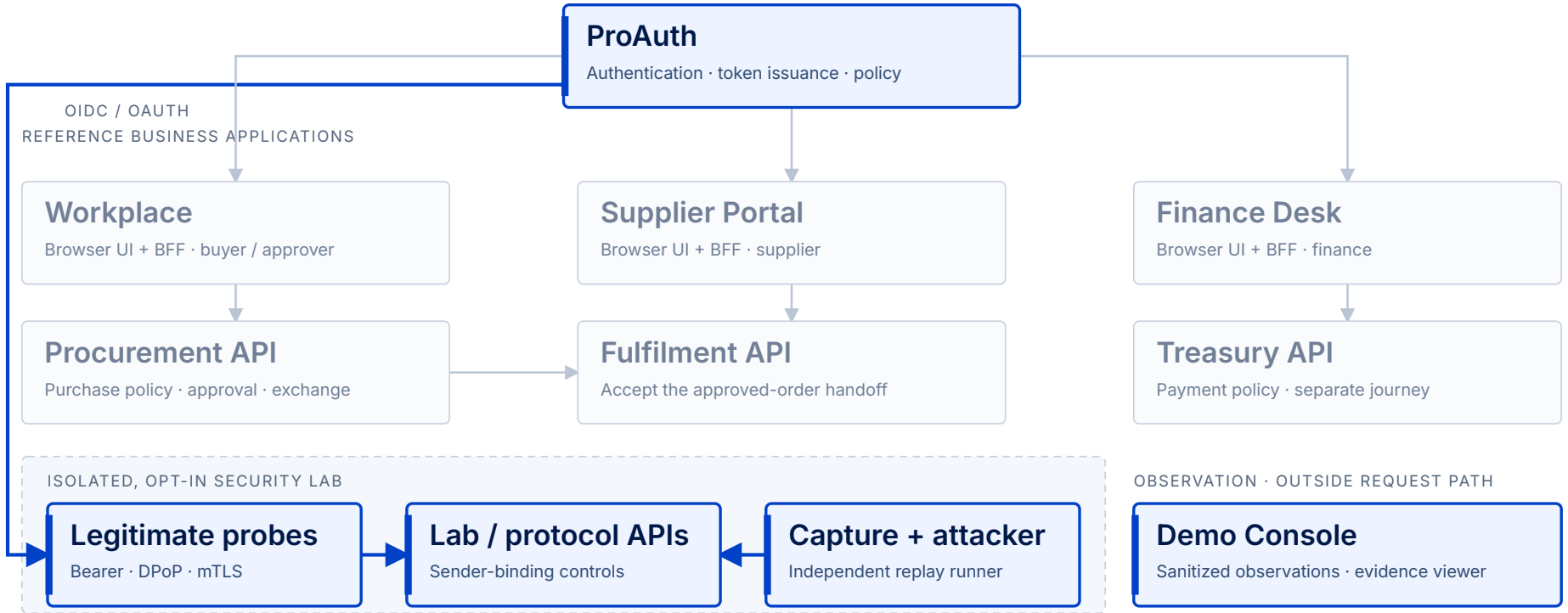
Replay fails

A copied bearer token works. Will a bound token?



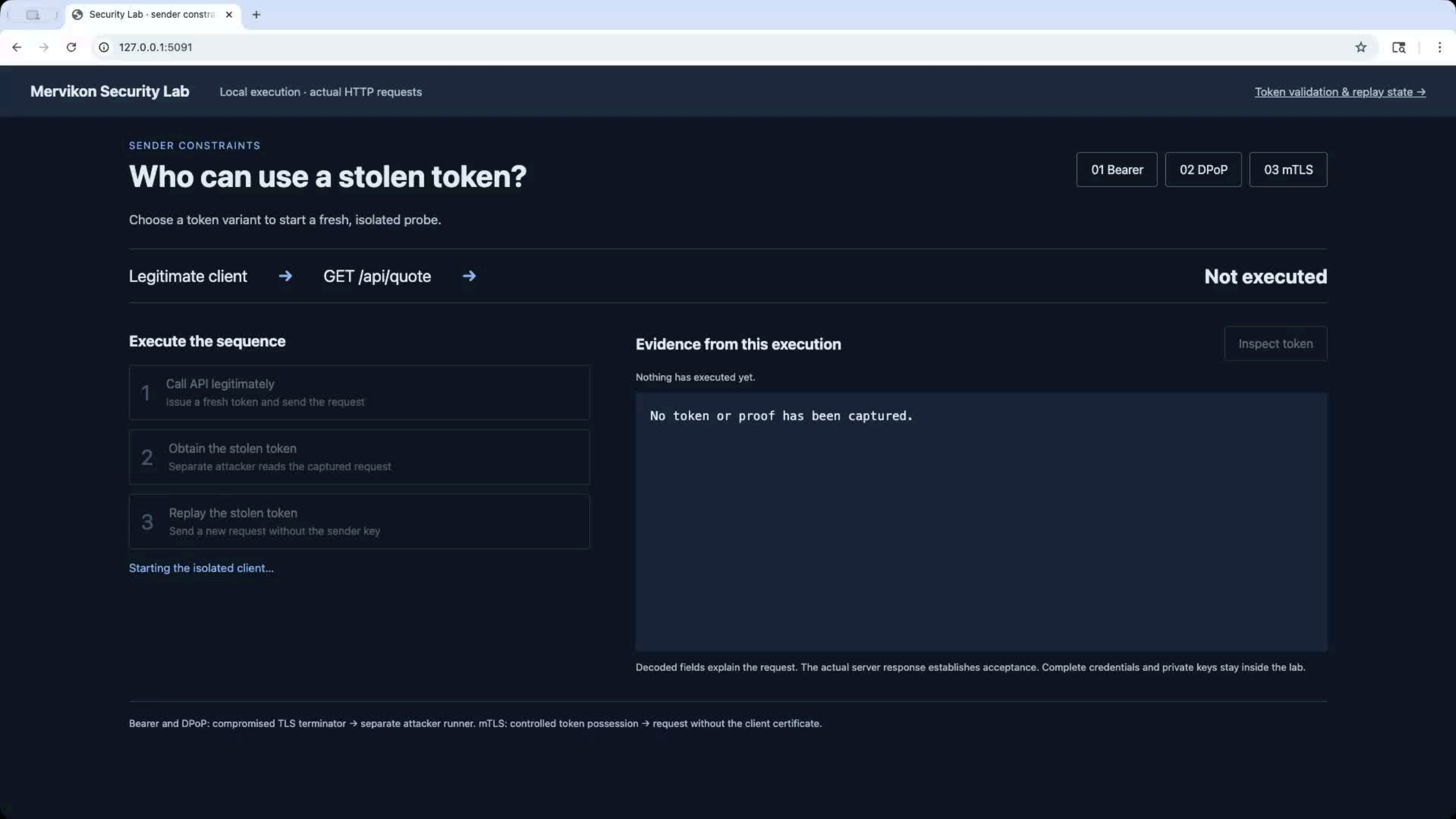
Where the stolen token is replayed

In this demo Grey components provide context



Independent replay against lab APIs. The Console shows completed bearer, DPoP and mTLS probes.

Reference implementation: architecture.md · conference.md · api-assurance.md. Logical overview; infrastructure and optional workers omitted.


Security Lab · sender constra
127.0.0.1:5091
Mervikon Security Lab
Local execution · actual HTTP requests
Token validation & replay state →
SENDER CONSTRAINTS
Who can use a stolen token?
01 Bearer
02 DPoP
03 mTLS
Choose a token variant to start a fresh, isolated probe.
Legitimate client → GET /api/quote →
Not executed
Execute the sequence
1 Call API legitimately
Issue a fresh token and send the request
2 Obtain the stolen token
Separate attacker reads the captured request
3 Replay the stolen token
Send a new request without the sender key
Starting the isolated client...
Evidence from this execution
Inspect token
Nothing has executed yet.
No token or proof has been captured.
Decoded fields explain the request. The actual server response establishes acceptance. Complete credentials and private keys stay inside the lab.
Bearer and DPoP: compromised TLS terminator → separate attacker runner. mTLS: controlled token possession → request without the client certificate.

Sender Constraint: Reduce Replay Value

RISK

Copied credentials

A leaked bearer token can be used by another caller.

CONTROL

Bound-key proof

The API also requires possession of the token’s bound key.

STILL REQUIRED

Correct enforcement

Operate keys, validation and replay state; enforce business policy.

> A stolen token alone becomes less useful. Live key misuse remains a risk.

A Copied Bearer Token Carries Its Authority

EXPOSURE

A token is copied

From a log, proxy trace or browser storage.

REPLAY

Another caller uses it

The API receives the same usable credential.

BUSINESS IMPACT

The attacker can act

The attacker can perform the operations that token allows.

RFC 6750 · RFC 9700

Proof Of Possession

01 **Access token** ✓

The authorization server grants access.

02 **Sender binding** ✓

The token names a key or certificate.

03 **Per-request proof** ✓

The caller proves possession of that key.

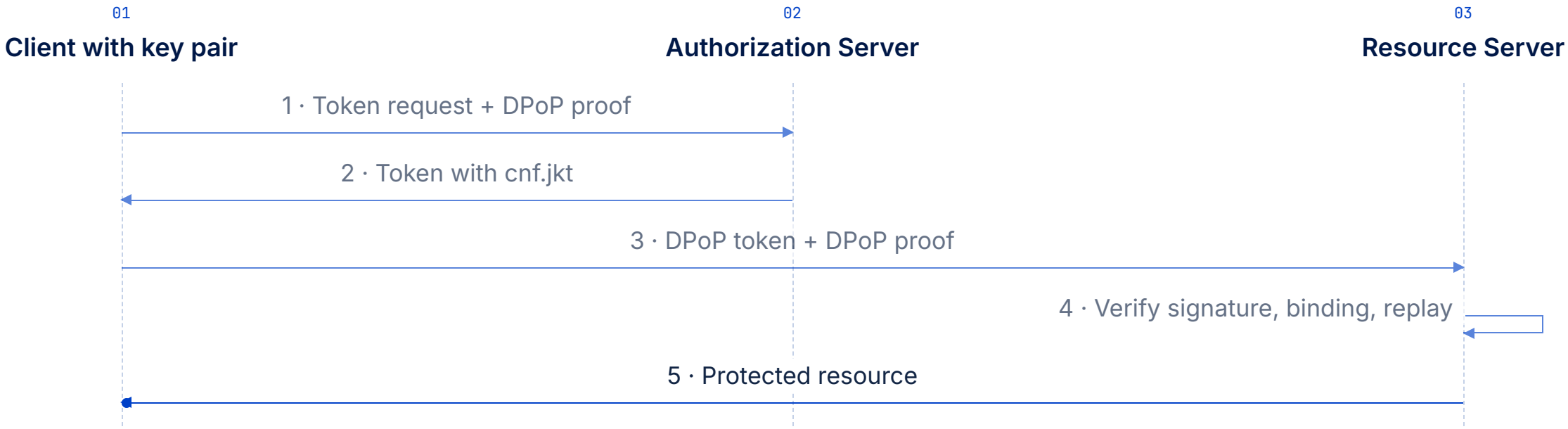
04 **Resource decision**

Accept only when token and proof match.

> Possession of the token alone is no longer sufficient.

RFC 9449 · RFC 8705

DPoP Flow



TRACE 5 / 5 Protected resource

RFC 9449

Two Headers, Two Different Jobs

Decoded binding excerpts · placeholders · not a complete wire message

```
Authorization: DPoP eyJhbGciOi...
DPoP: eyJ0eXAiOiJkcG9wK2p3dCIs...
```

```
{
  "token": { "cnf": { "jkt": "public-key-thumbprint" } },
  "proof": {
    "htm": "GET",
    "htu": "https://api.example/payments",
    "ath": "access-token-hash",
    "jti": "unique-proof-id", "iat": 1793441700
  }
}
```

RFC 9449

DPoP Proof JWT Anatomy

DPoP proof `header.payload.signature`

Separate from the access token

01 Header

`typ` `alg` `jwk`

02 Payload

`htm · htu` `ath` `jti · iat`

03 Signature

`header` . `payload` SIGNED TOGETHER

The verification key

Public JWK → thumbprint → `cnf.jkt`

Connects the proof to the access token.

`typ` proof type · `alg` signing algorithm

This request, this proof

`htm · htu`	→ Method + endpoint
`ath`	→ Access-token hash
`jti · iat`	→ Replay check + time

`nonce` also included when required

The tamper check

Private key **signs**

Public JWK **verifies**

Change header or payload?

The signature no longer verifies.

RFC 9449

Copied Proof, Different Endpoint

SIGN | ACCEPT | COPY | REPLAY | CHECK

ATTACKER · NO PRIVATE KEY

Copied token + proof

STOLEN DPoP PROOF · UNMODIFIED

HEADER . PAYLOAD . SIGNATURE

Decoded proof payload · signed values stay unchanged

`htm · htu`	**GET https://api.example/payments**
`jti · iat`	P-17 · within accepted time window
`ath`	Hash of the accompanying access token

Header: public `jwk` · Signature: client key · Token binding: `cnf.jkt`

P-17 →

ACTUAL REQUEST · api.example

GET /reports

RESOURCE SERVER · REQUEST BINDING

Signature + key / token binding	✓ VALID
Signed `htm / htu` vs. request	≠ MISMATCH

SIGNED URI /payments

ACTUAL URI /reports

REJECT · wrong endpoint

05 The signature is valid. The signed URI does not match the request.

Changing htu to /reports would invalidate the signature. The attacker cannot re-sign.

Assume both paths are allowed by token audience / scope. A bearer token alone has no per-request URI binding.

RFC 6750 · RFC 9449 §4.3

Same Endpoint, Already-Used Proof

SIGN | ACCEPT | COPY | REPLAY | CHECK

ATTACKER · NO PRIVATE KEY

ACTUAL REQUEST · api.example

Copied token + proof

STOLEN DPoP PROOF · UNMODIFIED

HEADER . PAYLOAD . SIGNATURE

Decoded proof payload · signed values stay unchanged

htm · htu	GET https://api.example/payments
jti · iat	**P-17 · within accepted time window**
ath	Hash of the accompanying access token

Header: public `jwk` · Signature: client key · Token binding: `cnf.jkt`

P-17 →

GET /payments

RESOURCE SERVER · REPLAY CACHE ENABLED

Signature + key / token binding ✓ VALID

Signed `htm / htu` vs. request ✓ MATCH

SEEN PROOFS · /payments

P-17 · expires with proof window HIT → REJECT

REJECT · proof already used

05 The signed values match. The replay cache rejects proof P-17.

API responsibility: track proof jti values for the accepted window; share state across replicas.

The access token is reusable. Each request needs a fresh proof; a signature check alone cannot detect reuse.

RFC 9449 §11.1 — proof replay and server state

DPoP Checks

HTTP REQUEST TO THE RESOURCE SERVER

```
Authorization: DPoP <access_token>
DPoP: <signed_proof_jwt>
```

Access token

`cnf.jkt`	public-key-thumbprint

Validate token or introspection first.
Then enforce API authorization.

DPoP proof

`jwk + signature`	sender key
`htm · htu · ath`	this request
`iat · jti · nonce*`	freshness

04 / FRESHNESS

Check iat, jti and a server nonce when required.

With single-use enforcement, atomically reject seen proof IDs.

* Nonce when required. htu excludes query/fragment; the body is not bound. Live key misuse remains possible.

RFC 9449

Follow The Key Through The Lifecycle

01 **Authorization code** ✓

dpop_jkt binds redemption to the intended key.

02 **Access token** ✓

The API requires proof of the token’s bound key.

03 **Public-client refresh token** ✓

Renewal requires the DPoP key bound at issuance.

04 **Downstream exchange**

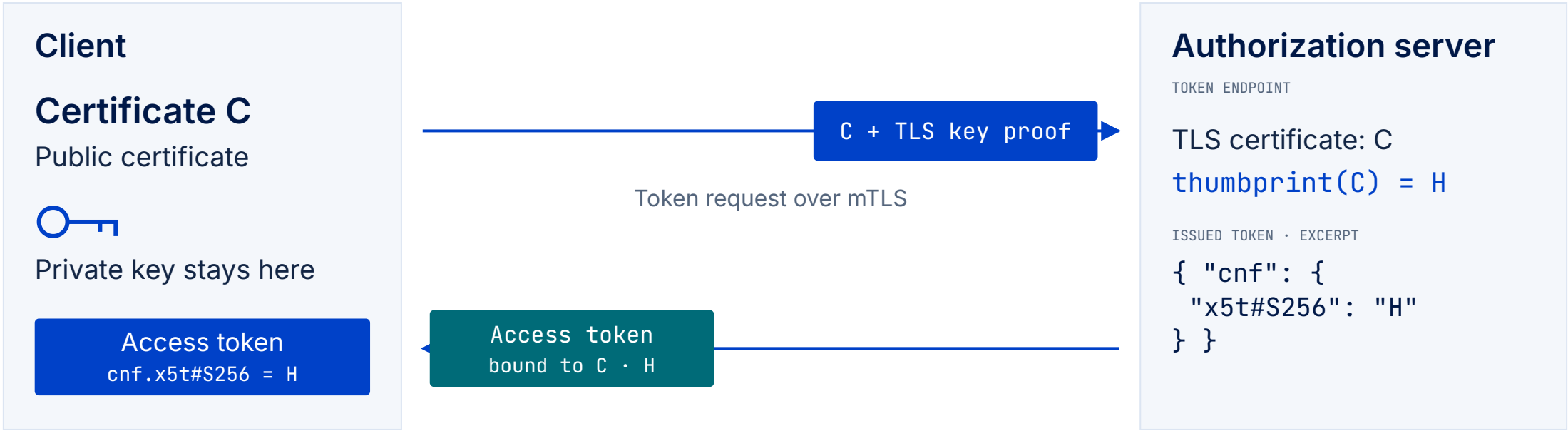
Policy can issue a new token bound to the next service’s key.

RFC 9449 · RFC 8693

mTLS: Get A Certificate-Bound Token

1 / ISSUE THE CERTIFICATE-BOUND TOKEN

CERTIFICATE + KEY | PROVE THROUGH TLS | BIND AT THE AS | RETURN TOKEN



The client receives an access token bound to C. The private key never travels.

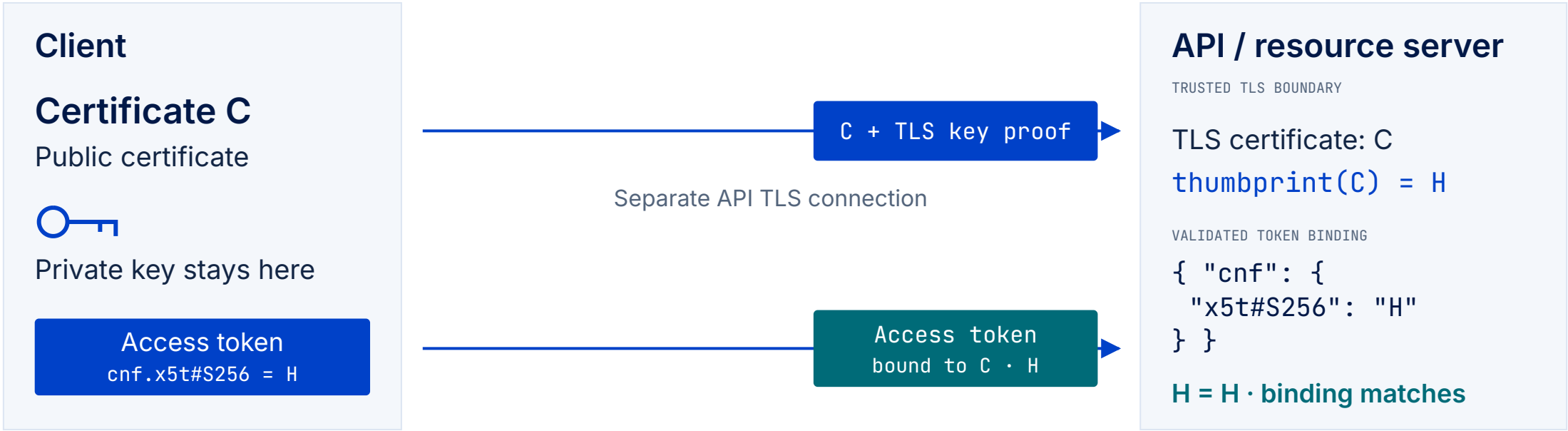
Decoded binding excerpt. H abbreviates the base64url SHA-256 hash of the DER certificate.

RFC 8705 §3

mTLS: Check The Binding At The API

2 / USE THE TOKEN AT THE RESOURCE SERVER

BOUND TOKEN | PROVE THROUGH TLS | PRESENT TOKEN | COMPARE AT THE API



Compare the TLS certificate thumbprint with the validated token binding.

A matching binding is one check. Token validation and business authorization still apply.

RFC 8705 §3

mTLS Replay Test

01 **Matching certificate** ✓

Token binding matches the client certificate → accept.

02 **Token alone** ✓

No corresponding client certificate → reject.

03 **Wrong certificate** ✓

Certificate thumbprint differs → reject.

04 **Trusted TLS boundary**

Verify the certificate at the API or trusted TLS terminator.

> The caller must possess the private key corresponding to the bound certificate.

RFC 8705

Which Sender Proof Can You Operate?

OPERATIONAL QUESTION	DPOP	CERTIFICATE-BOUND TOKENS
Where is possession proved?	Signed JWT at application layer	Client-authenticated TLS connection
What must the client operate?	Signing key + fresh proofs	Certificate + private key
What must the API operate?	Proof checks; replay state for strict single use	Certificate binding at trusted TLS boundary
What shapes the choice?	Client runtime and key access	Certificate lifecycle and proxy topology

RFC 9449 · RFC 8705

Accepting An API Request

01 **Trusted access token** ✓

Validate issuer, key, algorithm and lifetime; or use trusted introspection.

02 **Intended resource** ✓

Require the API audience and applicable access-token profile.

03 **Sender proof** ✓

Validate the proof and its binding to this token and request.

04 **Business authorization**

Enforce scopes, tenant, ownership and operation policy.

> A valid sender proof does not make an untrusted token valid.

RFC 9068 §4 · RFC 7662 · RFC 9449 §7

Validation Choices

Local JWT Validation

- Local validation
- Standard claims
- Revocation needs extra state

Introspection

- Opaque or JWT tokens
- Central status check
- Cache controls freshness

RFC 9068 · RFC 7662 · RFC 7009

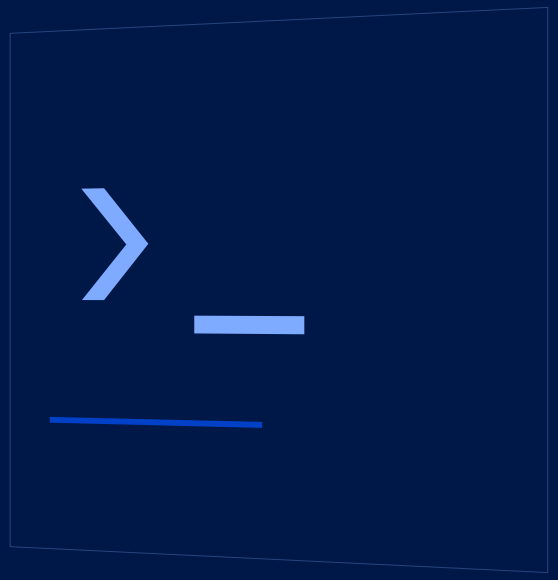
Browser Apps

Move the tokens. Keep the experience.

03

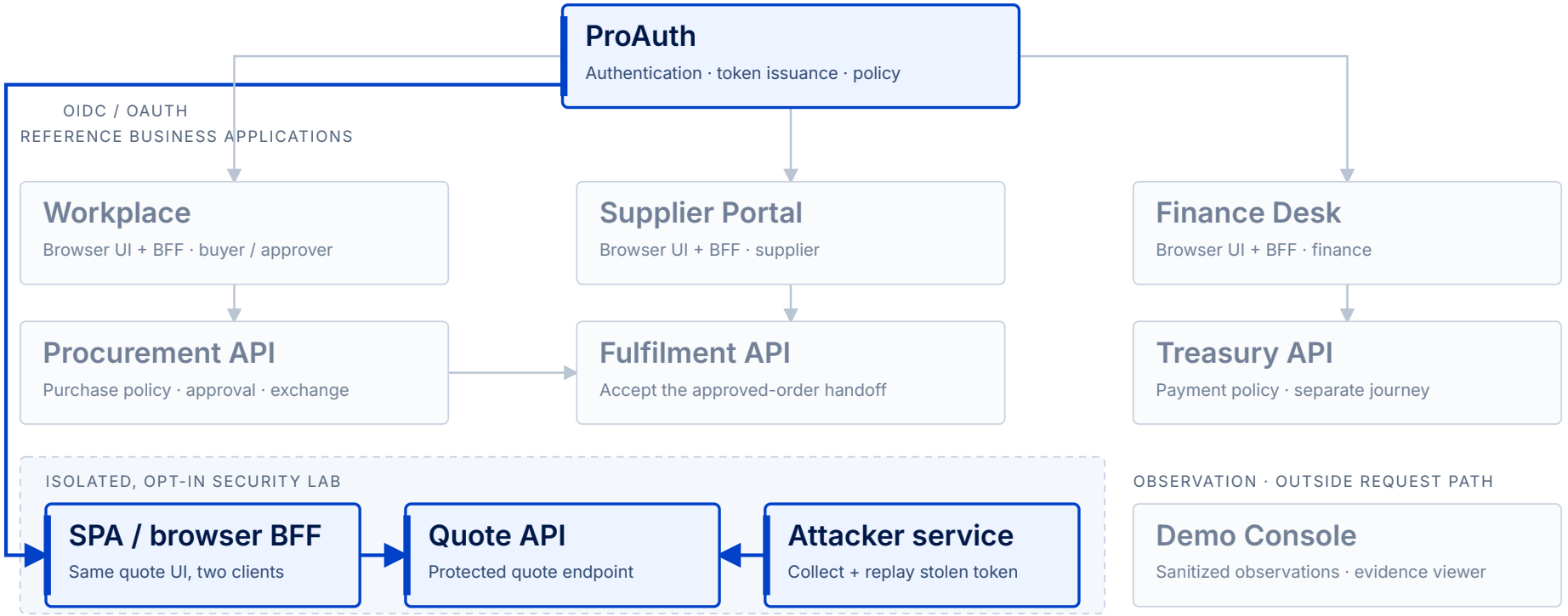
Direct SPA vs BFF

Find the token. Then move the boundary.



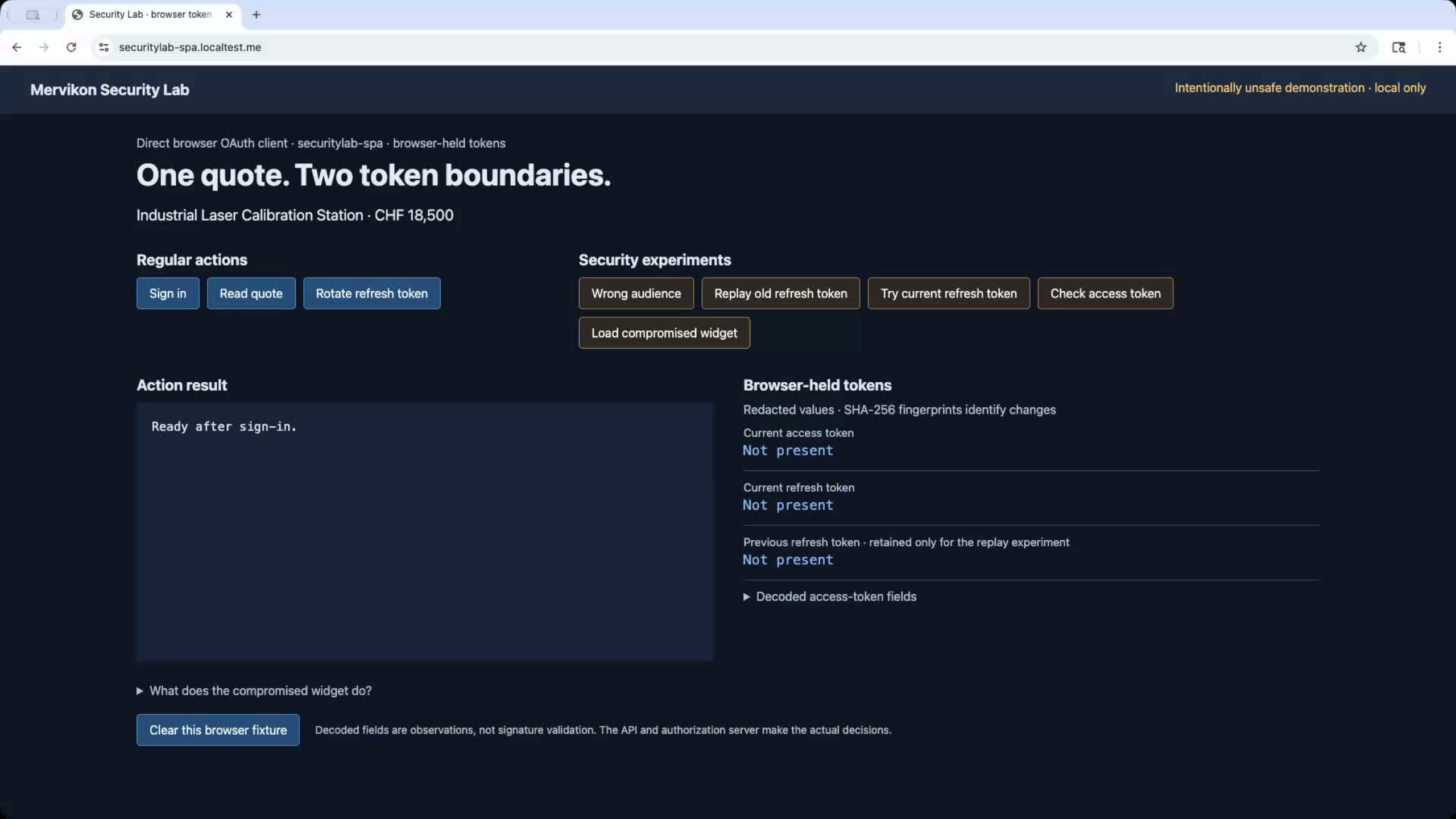
Same quote. Different token boundary.

In this demo Grey components provide context



The same quote and compromised widget. Move OAuth tokens from the SPA into a BFF.

Reference implementation: architecture.md · conference.md · api-assurance.md. Logical overview; infrastructure and optional workers omitted.


Security Lab · browser token
securitylab-spa.localtest.me
Mervikon Security Lab
Intentionally unsafe demonstration · local only
Direct browser OAuth client · securitylab-spa · browser-held tokens
One quote. Two token boundaries.
Industrial Laser Calibration Station · CHF 18,500
Regular actions
Sign in
Read quote
Rotate refresh token
Security experiments
Wrong audience
Replay old refresh token
Try current refresh token
Check access token
Load compromised widget
Action result
Ready after sign-in.
Browser-held tokens
Redacted values · SHA-256 fingerprints identify changes
Current access token
Not present
Current refresh token
Not present
Previous refresh token · retained only for the replay experiment
Not present
Decoded access-token fields
What does the compromised widget do?
Clear this browser fixture
Decoded fields are observations, not signature validation. The API and authorization server make the actual decisions.

Browser Boundary: Two Different Outcomes

RISK

Token extraction

The direct SPA exposes an access token to the compromised widget.

CONTROL

BFF boundary

This probe captures no OAuth token from the BFF application.

REMAINING RISK

Session misuse

The same-origin action still succeeds; returned data can be read.

One Script, Two Attacker Outcomes

01 **Code exchange** ✓

S256 PKCE protects code redemption.

02 **Token issued** ✓

The direct SPA now has a readable access token.

03 **Extract and replay** ✓

Malicious JavaScript copies a token for use elsewhere.

04 **Act from the origin**

The script calls APIs and reads responses as the user.

> Moving tokens changes extraction risk. It does not remove malicious JavaScript.

RFC 10017 · RFC 9700

Three Browser Patterns

ARCHITECTURE	TOKENS READABLE BY JAVASCRIPT	API REQUESTS
Full BFF	Neither access nor refresh token	Proxied by backend
Token-mediating backend	Access token; refresh token stays backend	Browser calls API
Browser-only OAuth client	Access token; refresh token if issued	Browser calls API

RFC 10017 §6 · browser architectures

Move The Token Boundary

SAME UI · SAME RESOURCE API · DIFFERENT TOKEN EXPOSURE

Session cookie →

API access token →

BROWSER

Application UI

No readable OAuth tokens

HttpOnly session cookie

BACKEND FOR FRONTEND

OAuth + proxy

Obtain and renew tokens

Proxy approved API calls

THIS DEMO

OAuth tokens

Backend token store

RESOURCE API

Business service

Validate access token

Authorize operation

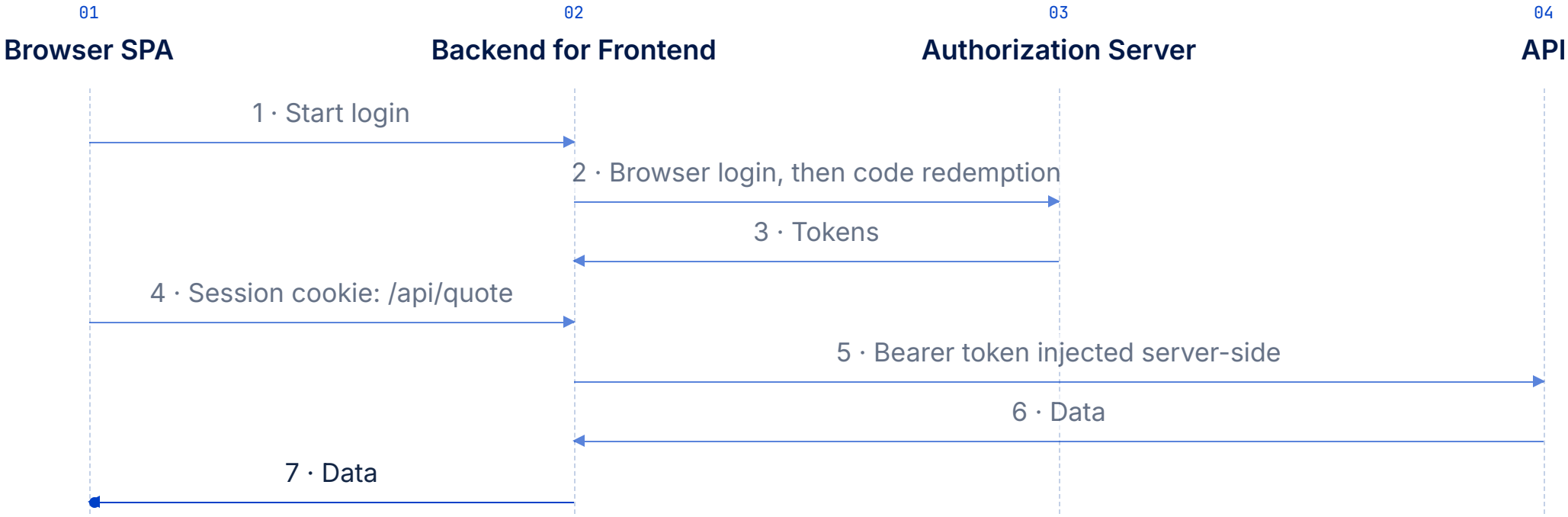
Return protected data

> The browser uses its session; the BFF attaches the API token.

XSS can still act through the session and read returned data. Keep CSRF and XSS defenses.

Browser-based apps · RFC 10017 · ProAuth BFF

Full BFF



TRACE 7 / 7 **Data**

Browser-based apps · RFC 10017 · ProAuth BFF

Who Operates The Browser-Facing Backend?

01 **BFF responsibilities** ✓

OAuth login, renewal, session handling and API proxying.

02 **One deployment** ✓

An application backend can perform all these roles.

03 **Separate components** ✓

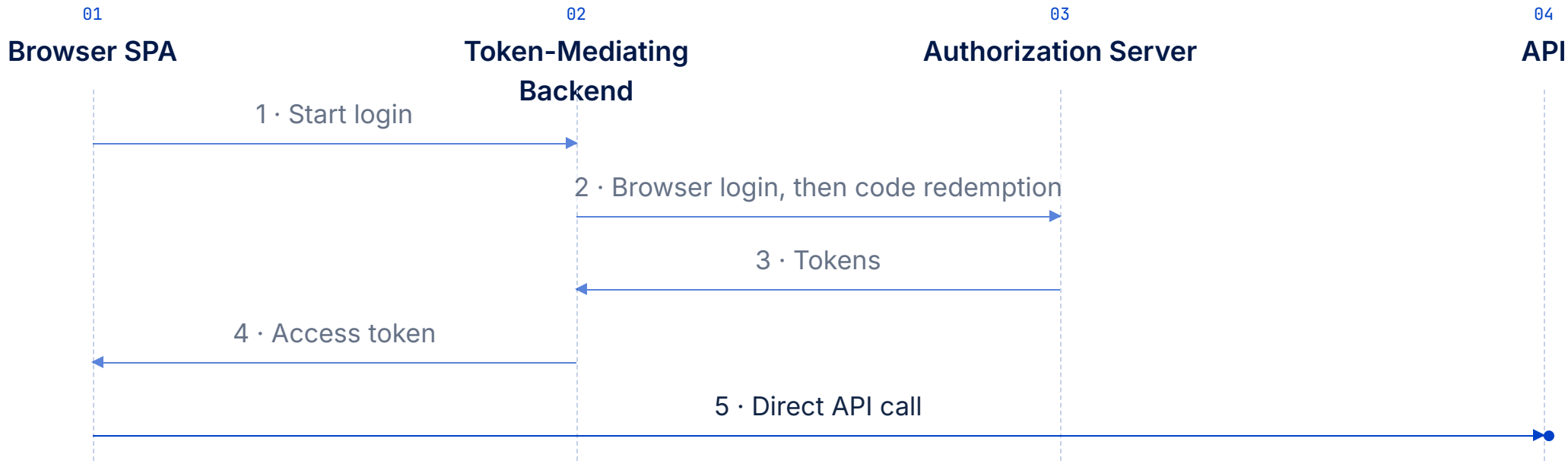
An OAuth/session service and an API proxy can share the work.

04 **Token Handler terminology**

Here: that separation while retaining the BFF token boundary.

BFF security model · RFC 10017 §6.1

Token-Mediating Backend



TRACE 5 / 5 **Direct API call**

Browser-based apps · RFC 10017

BFF: Protect The Session Too

- Protect the session cookie: Secure, HttpOnly, SameSite and host scope
- Reject cross-site actions with explicit CSRF defenses
- Prevent script compromise: XSS can still act and read data
- Restrict the proxy to approved API destinations

RFC 10017 §6.1.3 · BFF security considerations

Choose The Browser Boundary

DEPLOYMENT SITUATION	DIRECTION TO EVALUATE	REMAINING RESPONSIBILITY
APIs can be proxied; sensitive application data	Full BFF	Session security, proxy operations, XSS
Direct browser-to-API calls are required	Token-mediating backend	Access tokens remain exposed to JavaScript
No backend is viable	Browser-only OAuth client	Own the complete browser token lifecycle

> Evaluate a full BFF first. Make unavoidable token exposure explicit.

RFC 10017

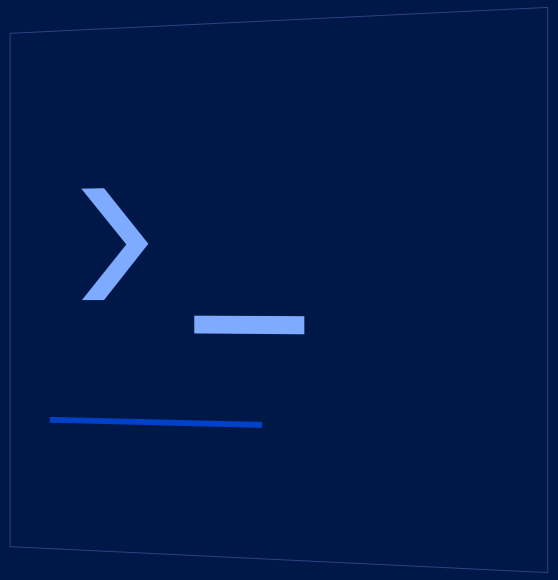
Non-Human Access

04

Name the actor. Narrow the next hop.

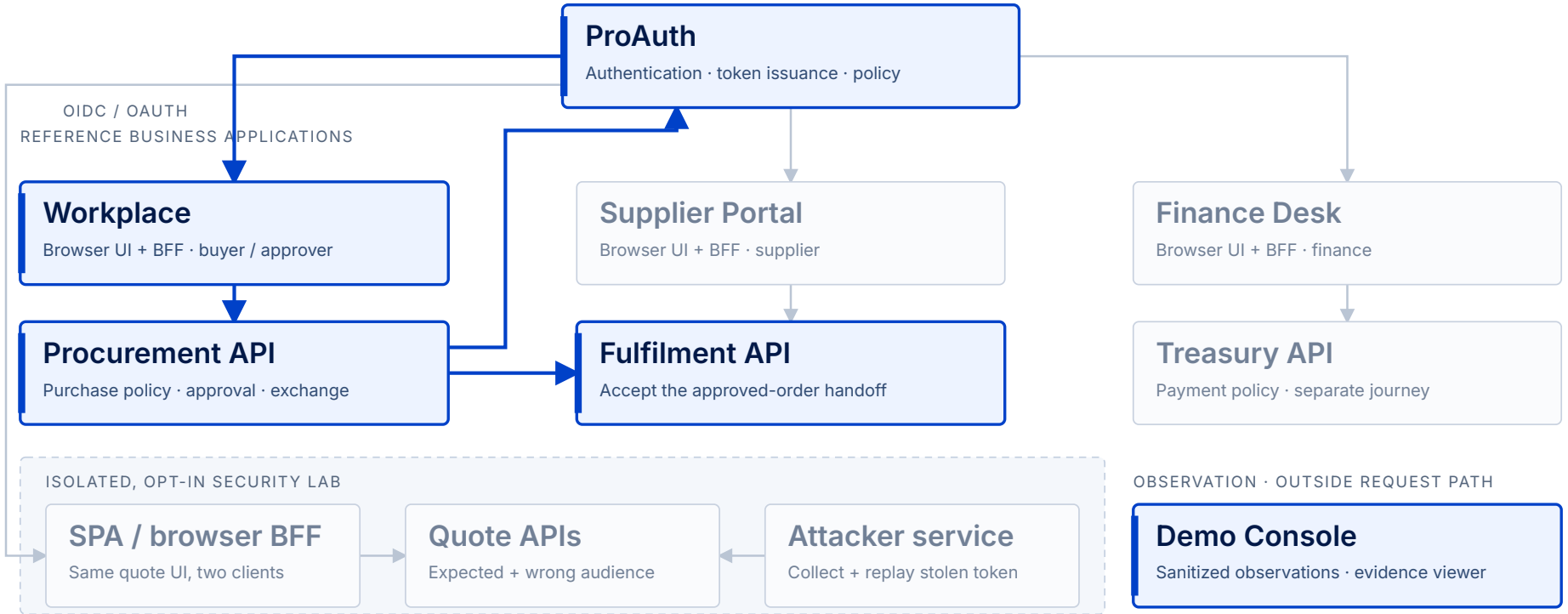
Delegation chain

One approved order. Two APIs. Which token should the second accept?



The next hop is a different audience

In this demo Grey components provide context



Workplace → Procurement → Fulfilment. Procurement exchanges at ProAuth using its own identity and key.

Reference implementation: architecture.md · conference.md · api-assurance.md. Logical overview; infrastructure and optional workers omitted.

Mervikon Group

WORKPLACE

Procurement 01

Equipment catalogue 02

PURCHASING

Equipment for the work ahead.

Request equipment, review your purchases and approve within your authority.

Sign in to see your purchases. Access is checked by the Procurement API.

Your purchases

0 visible

EQUIPMENT	VALUE	STATUS

Equipment catalogue

CHF · demonstration fixtures

01 **Industrial Laser Calibration Station**
Standard approval · separation of duties applies.
CHF 18'500 Request

02 **Mission-Critical Coffee Infrastructure**
Approval requires verified MFA or phishing-resistant authentication within five minutes.
CHF 24'900 Request

03 **Autonomous Office Assistant Robot**
CHF 42'000 Request

Delegation: Limit The Next Hop

RISK

Authority spreads

Forwarding broad user tokens expands where they can be used.

CONTROL

Explicit exchange policy

Grant the next API’s approved audience, action and sender binding.

STILL REQUIRED

Business checks

The API checks actor, organization, ownership and purchase state.

Name The Actor

01 Own behalf ✓

A service uses client credentials.

02 On behalf of a user ✓

An API exchanges a token for its downstream call.

03 Dynamic ecosystem

Discover metadata; establish trust through policy.

> “Machine-to-machine” is not one authorization pattern.

RFC 6749 · RFC 8693 · RFC 9728 · OpenID Federation

Three Separate Security Questions

QUESTION	CONTROL	WHAT IT ESTABLISHES
Which registered client?	private_key_jwt or mTLS authentication	Client identity at the AS
Which sender holds the key?	DPoP or certificate-bound tokens	Possession of the token’s bound key
Is this operation allowed?	AS policy + API authorization	Permitted authority and business action

RFC 9449 §2 · RFC 8705 §1 · OIDC client authentication

Strong Client Identity

- Prefer `private_key_jwt` or mTLS authentication where feasible
- Replace long-lived shared secrets with managed signing keys
- Plan key rotation and compromise recovery
- Limit each service’s scopes and audiences

RFC 9700 · RFC 8705 · OIDC Core

Give Each API Its Own Authority

Incoming token

```
{ "aud": "procurement",
  "scope": "procurement.write" }
```

Next operation

Fulfilment needs `fulfilment.create`.

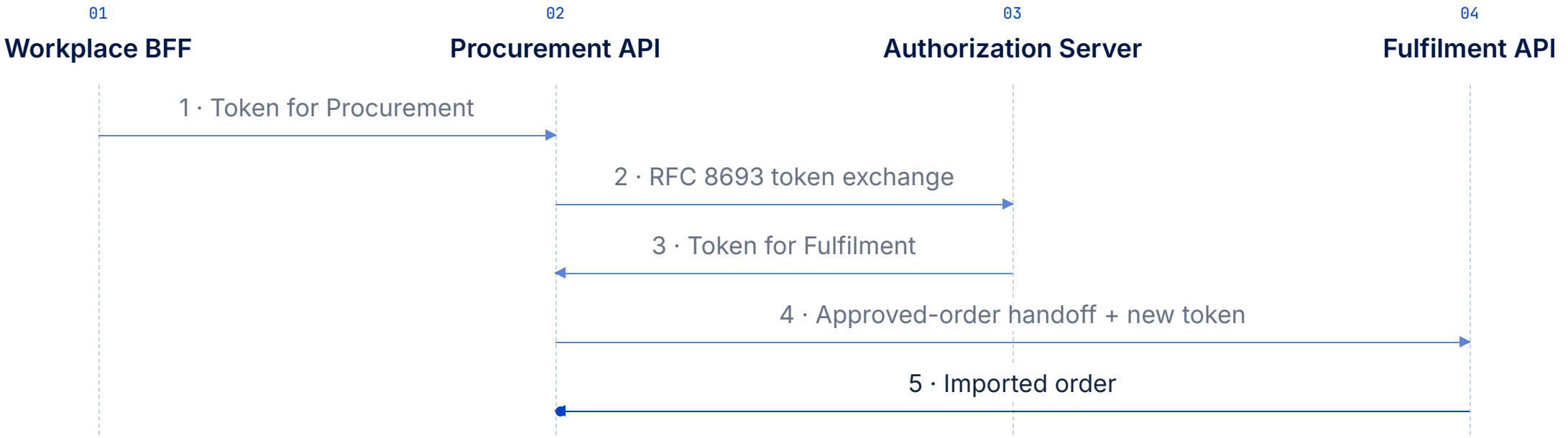
The incoming audience is wrong for this API.

> Request the intended resource; the API validates its own audience and authority.

Selected claims; identifiers shortened. Scope names across APIs are not a literal subset comparison.

RFC 8707 · RFC 9068 · RFC 8693

Token Exchange



TRACE 5 / 5 Imported order

RFC 8693

NON-HUMAN ACCESS

Policy Narrows The Next Hop

Workplace BFF → Procurement API → Fulfilment API

INCOMING SUBJECT TOKEN

aud	procurement
scope	procurement.read procurement.write
sub	user-123
cnf.jkt	workplace-key

RFC 8693

EXCHANGE

Authorization server

Authenticate caller
Evaluate delegation policy

NEW DOWNSTREAM TOKEN

aud	fulfilment
scope	fulfilment.create
sub	user-123
act.sub	procurement-client
cnf.jkt	procurement-key

Same subject. Named actor. New sender key.

Selected claims; identifiers shortened. This policy grants fulfilment.create; exchange does not automatically downscope.

RFC 8693 · RFC 8707 · RFC 9700

Exchange Request Data

Input

- Token-exchange `grant_type`
- `subject_token` + its type
- Target `resource` / `audience`
- Requested `scope`
- Caller authentication / actor context

Output

- Issued token + type metadata
- Policy-approved audience
- Policy-approved authority
- Subject / actor per policy

RFC 8693

Discovering An API Does Not Establish Trust

01 **Find the resource metadata** ✓

An API challenge can point to protected-resource metadata.

02 **Read the configuration** ✓

Discover supported authorization servers and capabilities.

03 **Apply the trust policy** ✓

Decide which issuers and resource relationships are acceptable.

04 **Scale partner onboarding**

Federation can validate trust chains under ecosystem policy.

RFC 9728 · OpenID Federation

Valid Token. Is This Order Allowed?

01 **API receives an authenticated call** ✓

The token, audience and sender checks have passed.

02 **Evaluate the business operation** ✓

May this actor fulfil PO-1042 for this organization now?

03 **Externalize when useful** ✓

AuthZEN standardizes requests to an authorization decision service.

04 **Enforce at the API**

Apply the decision; re-evaluate changing business conditions.

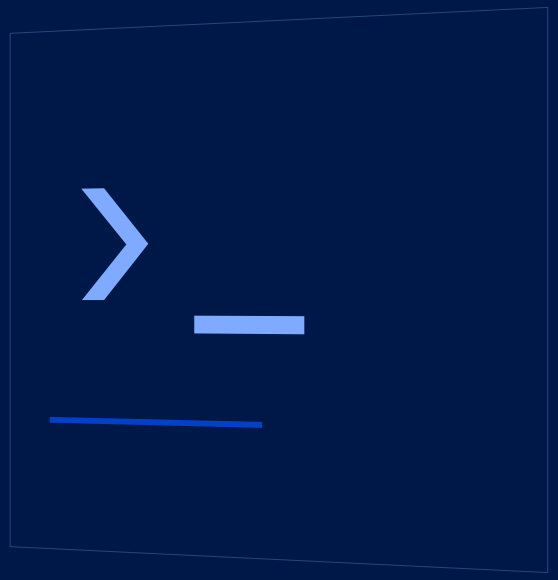
AuthZEN Authorization API

High-Assurance Request Flows

05

Protect sensitive actions and the authority behind them.

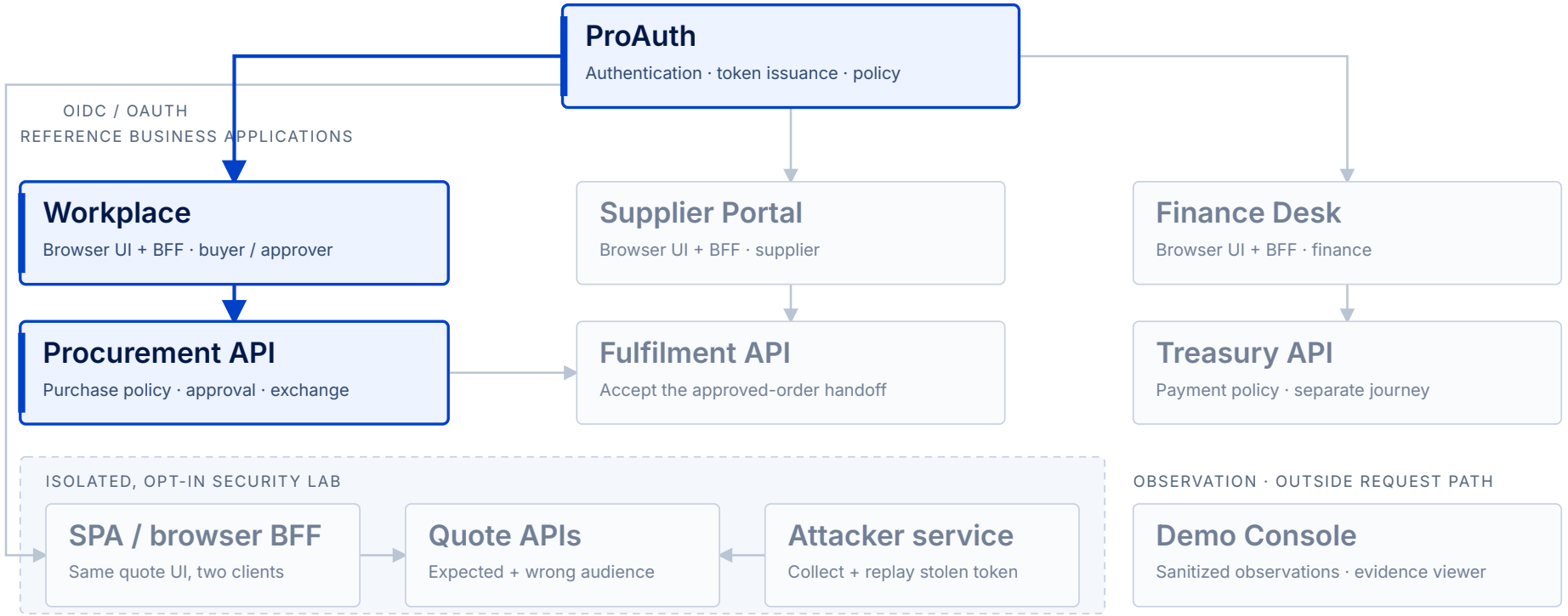
Signed in. Still asked to verify.



Attempt a high-value approval. Verify identity. Confirm the action again.

The API decides when to verify again

In this demo Grey components provide context



Procurement challenges the approval. Workplace returns to ProAuth, then asks for explicit confirmation.

Reference implementation: architecture.md · conference.md · api-assurance.md. Logical overview; infrastructure and optional workers omitted.

Mervikon Group
WORKPLACE

Procurement 01
Equipment catalogue 02

Procurement / Workplace

Sign in

PURCHASING

Equipment for the work ahead.

Request equipment, review your purchases and approve within your authority.

Sign in to see your purchases. Access is checked by the Procurement API.

Your purchases

0 visible

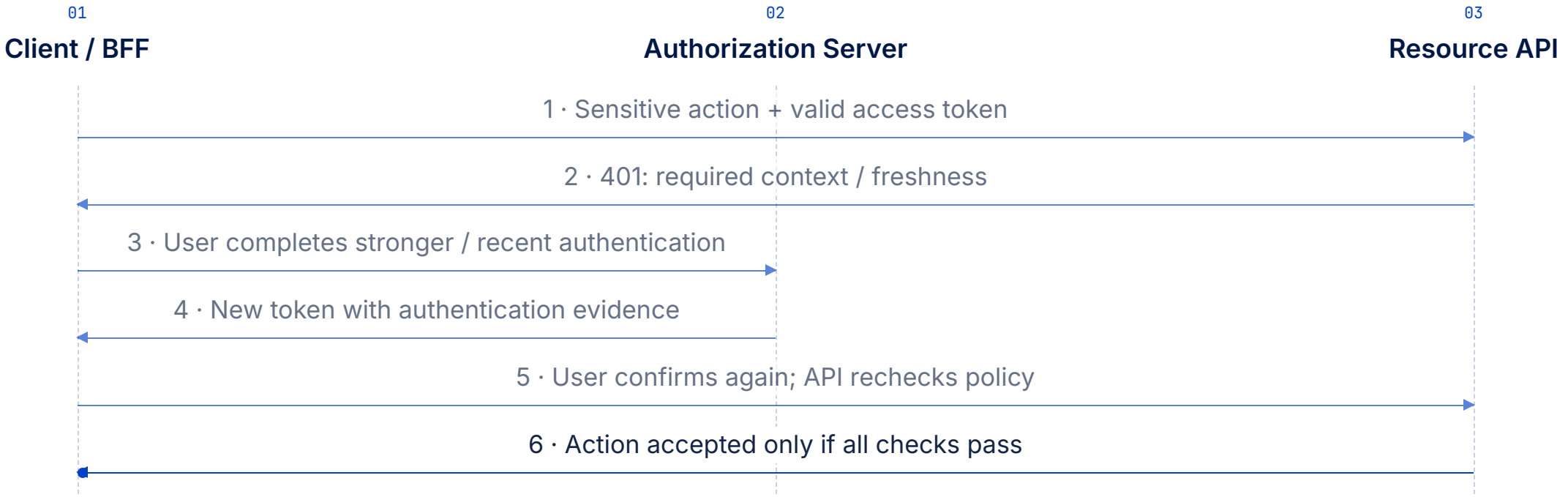
EQUIPMENT	VALUE	STATUS

Equipment catalogue

CHF · demonstration fixtures

01	Industrial Laser Calibration Station Standard approval · separation of duties applies.	CHF 18'500	Request
02	Mission-Critical Coffee Infrastructure Approval requires verified MFA or phishing-resistant authentication within five minutes.	CHF 24'900	Request
03	Autonomous Office Assistant Robot	CHF 42'000	Request

A Valid Token Can Still Need Step-Up



TRACE 6 / 6 **Action accepted only if all checks pass**

Check trusted acr and original auth_time. Refreshing a token does not refresh authentication.

RFC 9470 · step-up authentication challenge

Protect The Authorization Behind The Action

ILLUSTRATIVE BUSINESS REQUEST · SEPARATE FROM THE LIVE DEMO

> Allow fulfilment of approved order PO-1042 for organization ACME.

MEANING

What is allowed?

One action, one order, one organization.

INTEGRITY

What reached the AS?

The intended parameters, from the intended source.

ENFORCEMENT

What will the API do?

Apply the granted limits and current business rules.

When A Scope Is Too Coarse

Broad permission

`scope=fulfilment.create`

Which order? Which organization?

Structured request

```
{ "authorization_details": [{
  "type": "order_fulfilment",
  "actions": ["fulfil"],
  "order_id": "PO-1042",
  "organization": "ACME"
}] }
```

> RAR carries precise requested authority. The AS and API must enforce its meaning.

Illustrative application-defined type and fields; not a standard order schema or a complete request.

RFC 9396

Do These Details Belong In The Browser URL?

Authorization request excerpt

```
/authorize?client_id=workplace
 &response_type=code
 &scope=fulfilment.create
 &authorization_details=...
```

Decoded for teaching; other required parameters omitted.

Inside those details

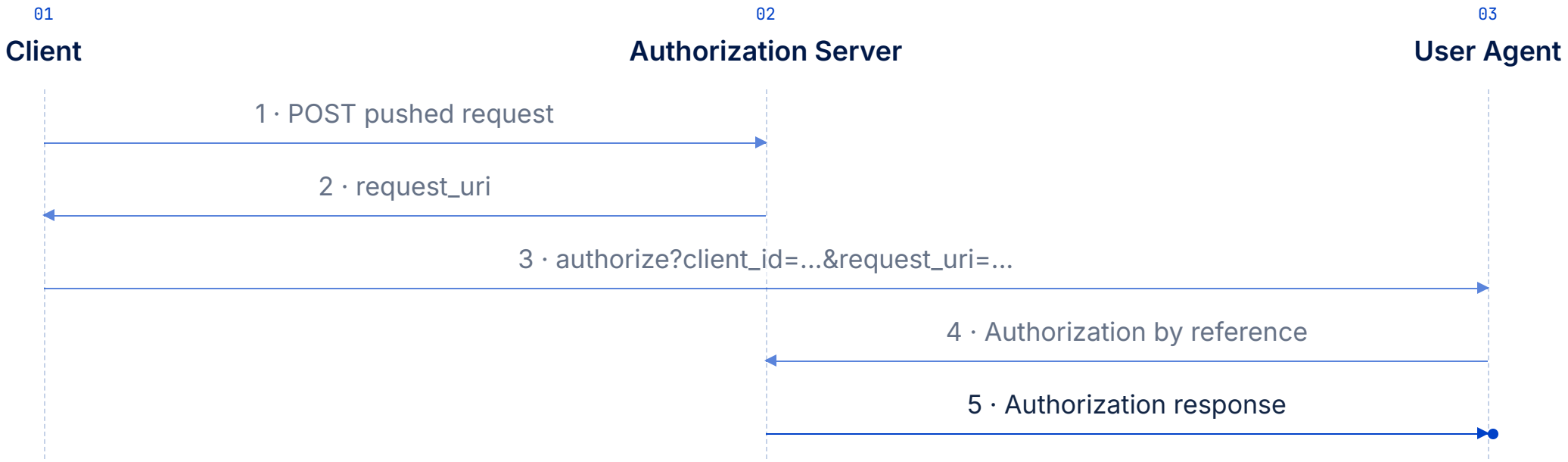
Order: **PO-1042**
Organization: **ACME**
Action: **fulfil**

Browser history, diagnostics and URL handling now touch business context.

> Push the detailed request to the AS. Let the browser carry a short-lived reference.

RFC 9126 · RFC 9101

PAR



TRACE 5 / 5 **Authorization response**

RFC 9126

After PAR

Back Channel

- Full request data
- Authenticate confidential clients
- Early validation
- Stored by AS

Front Channel

- `client_id`
- `request_uri`
- Less URL exposure
- Less tamper surface

RFC 9126

JAR: Verify Who Signed These Parameters

Client-signed request object

```
{ "iss": "workplace-client",
  "authorization_details": [{
    "type": "order_fulfilment",
    "order_id": "PO-1042",
    "organization": "ACME"
  }] }
```

Custom order schema; decoded payload excerpt, signature omitted.

Attempted edit

`order_id: PO-9999`

The signature no longer verifies.

Verify the expected signer and the unchanged request parameters.

> PAR protects direct submission. Add JAR when a signed request object is required.

RFC 9101 · RFC 9126

JARM: Protect The Authorization Response

REQUEST

Client → AS

PAR protects direct submission. JAR can protect a signed request object.

RESPONSE

AS → client

JARM wraps authorization response parameters in a signed JWT.

VALIDATION

Client checks

Trusted signature, issuer, audience, expiry and transaction binding.

JARM

FAPI: Make The Profile Enforceable

01 **PAR** ✓

Push and validate the authorization request.

02 **Code + S256 PKCE** ✓

Require the modern authorization code flow.

03 **Confidential client** ✓

Authenticate with private_key_jwt or mTLS.

04 **Sender constraint**

Bind access tokens with DPoP or mTLS.

Selected FAPI 2.0 controls; conformance requires the complete profile.

RFC 9700 · FAPI 2.0

FAPI Still Needs An Operating Model

API AUTHORIZATION

May this actor act?

Check the organization, object ownership and current order state.

KEY OPERATIONS

Who handles compromise?

Rotate keys and certificates; monitor failures and revoke access.

GOVERNANCE

Whom do we trust?

Define participant policy and verify the complete profile.

FAPI 2.0

High Assurance: Protect The Intended Authority

RISK

Ambiguous or altered requests

Rich business authorization crosses several components.

CONTROL

Express and protect

RAR for detail; PAR for direct submission; signed objects when needed.

STILL REQUIRED

Enforce and operate

API policy, key lifecycle and ecosystem governance remain essential.

Decision Matrix

Turn protocol choices into an implementation plan.

06

DECISION MATRIX

Fix The Baseline First

IF YOU FIND…	CHANGE	CHECK BEFORE ROLLOUT
Implicit flow or collected user passwords	Code + S256 PKCE; redirect-based login	Client migration and AS support
Loose redirect validation	Exact matching	Registered URIs; native loopback exception
Multiple issuers without correlation	Bind expected issuer to the transaction	Validate response issuer before redemption
Public-client refresh tokens without replay protection	Rotation or sender constraint	Family revocation or bound-key lifecycle

RFC 9700 · RFC 9207

DECISION MATRIX

Match The Architecture To The Exposure

IF YOU FIND...	EVALUATE	MAIN DEPENDENCY
Sensitive browser tokens readable by scripts	Full BFF	API proxying and session operations
Leaked tokens replayable from another caller	DPoP or certificate-bound tokens	Key lifecycle and complete API enforcement
Long-lived shared client credentials	Asymmetric client authentication	Key registration, rotation and revocation
User authority forwarded across API hops	Policy-controlled token exchange	Explicit delegation rules and downstream checks

RFC 10017 · RFC 9449 · RFC 8705 · RFC 8693

DECISION MATRIX

Recognize When You Need More

YOUR TEAM SAYS…	EVALUATE
“Business authorization details pass through browser URLs.”	PAR
“The recipient must verify a client-signed request object.”	JAR
“Our ecosystem requires a signed authorization response.”	JARM
“A scope cannot express this order, action or limit.”	RAR
“Every integration needs the same high-security protocol baseline.”	FAPI 2.0
“Trust onboarding across partner organizations will not scale.”	Federation

RFC 9126 · RFC 9101 · RFC 9396 · FAPI 2.0

DECISION MATRIX

Migration Backlog

01 **Inventory** ✓

List clients, grants and token locations.

02 **Baseline** ✓

Fix PKCE, redirects, issuer checks and refresh handling.

03 **Browser boundary** ✓

Move OAuth tokens behind a BFF where viable.

04 **Sender constraint** ✓

Reduce the value of copied tokens.

05 **Downstream access** ✓

Narrow agent and API authority.

06 **High-value flows**

Apply request protection and ecosystem profiles.

RFC 9700 · RFC 9449 · RFC 8693 · Browser-based apps · RFC 10017

TAKEAWAYS

The Token Is Valid. Should This Request Be Allowed?

A valid token is the start of the decision.

Constrain the sender.

Reduce exposure.

Limit authority.

QUESTIONS & CONTACT

Let’s continue the conversation

Marc Müller

Principal Consultant

marcromanmueller

marc-mueller

@muellermarc

4tecture GmbH

Industriestrasse 25

CH-8604 Volketswil

Consulting · Architecture · Training

www.4tecture.ch

www.proauth.net

TAKE THE SLIDES WITH YOU

www.4tecture.ch/events/codetalks2

6-identityprotocols

Thank you.

Design for token context.

Audience. Sender. Boundary. Authority.

TAKE THE SLIDES WITH YOU

www.4tecture.ch/events/codetalks2

6-identityprotocols

Appendix

A field guide to the standards behind the architecture.

APPENDIX

OAuth 2.0

- **Means:** Authorization framework
- **Benefit:** Delegated API access without sharing passwords
- **How:** Clients obtain access tokens for APIs after an authorization grant
- **Use when:** An app needs API access on behalf of a user, service, or workload

RFC 6749

APPENDIX

OIDC

- **Means:** OpenID Connect
- **Benefit:** Standardized user sign-in and identity claims
- **How:** Adds ID tokens, UserInfo, discovery, scopes, and claims to OAuth
- **Use when:** A client needs to authenticate a user, not only call an API

OIDC Core

APPENDIX

JWT, JWS, JWE

- **JWT:** JSON Web Token, a compact claims container
- **JWS:** Content protected by a digital signature or MAC
- **JWE:** Encrypted content with integrity protection
- **Use when:** Tokens or protocol messages need portable cryptographic protection

RFC 7519 · RFC 7515 · RFC 7516

PKCE

- **Means:** Proof Key for Code Exchange
- **Benefit:** Protects authorization codes from interception
- **How:** Send an S256 challenge; redeem the code with its verifier
- **Use when:** Authorization code flow is used, especially for public clients

RFC 7636 · RFC 9700

APPENDIX

RFC 9207 Issuer Identification

- **Means:** Authorization Server Issuer Identification
- **Benefit:** Helps prevent OAuth mix-up attacks
- **How:** Authorization responses include `iss`; clients compare it with expected issuer
- **Use when:** Clients interact with one or more authorization servers

RFC 9207

APPENDIX

OAuth Security BCP

- **Means:** OAuth 2.0 Security Best Current Practice
- **Benefit:** Modern baseline for safer OAuth deployments
- **How:** Recommends code + PKCE, exact redirects, issuer checks, replay protection, and sender constraint
- **Use when:** Designing or reviewing OAuth deployments in 2026

RFC 9700

APPENDIX

OAuth 2.1

- **Means:** Consolidation of OAuth 2.0 with current best practices
- **Benefit:** Removes old patterns from the mental default
- **How:** Centers authorization code + PKCE and drops implicit and password grants
- **Use when:** Explaining where modern OAuth is heading

OAuth 2.1 draft

Browser-Based Apps

- **Means:** OAuth guidance for browser applications
- **Benefit:** Names malicious JavaScript as a central threat
- **How:** Compares browser-only clients, token-mediating backends, and full BFFs
- **Use when:** Deciding where browser tokens should live

Browser-based apps · RFC 10017

BFF

- **Means:** Backend for Frontend
- **Benefit:** Moves OAuth tokens out of browser JavaScript
- **How:** Backend handles OAuth and proxies APIs; JavaScript receives no readable tokens
- **Use when:** A browser app handles sensitive data and APIs can be proxied

Browser-based apps · RFC 10017 · ProAuth BFF

APPENDIX

DPoP

- **Means:** Demonstrating Proof of Possession
- **Benefit:** Stolen access token alone is not enough
- **How:** Token is bound to a public key thumbprint; each request carries a signed proof
- **Use when:** Sender constraint is needed but mTLS is impractical

RFC 9449

mTLS

- **Means:** Mutual TLS client authentication and certificate-bound tokens
- **Benefit:** Strong sender constraint for controlled server environments
- **How:** API checks certificate binding from JWT claims or introspection
- **Use when:** Backend services can operate client certificates safely

RFC 8705

PAR

- **Means:** Pushed Authorization Requests
- **Benefit:** Keeps rich authorization request data out of the browser URL
- **How:** Client posts request details to the authorization server and receives `request_uri`
- **Use when:** Authorization requests are high-value, long, sensitive, or FAPI-style

RFC 9126

JAR

- **Means:** JWT-Secured Authorization Request
- **Benefit:** Integrity and optional confidentiality for authorization request parameters
- **How:** Client sends request parameters inside a signed, optionally encrypted JWT
- **Use when:** Request parameters must not be changed silently in the front channel

RFC 9101

JARM

- **Means:** JWT-Secured Authorization Response Mode
- **Benefit:** Integrity and optional confidentiality for authorization responses
- **How:** Authorization response parameters are returned inside a signed, optionally encrypted JWT
- **Use when:** The response itself needs cryptographic protection

RAR

- **Means:** Rich Authorization Requests
- **Benefit:** Expresses structured authorization details beyond flat scopes
- **How:** Clients send `authorization_details` with typed business objects
- **Use when:** Scopes are too coarse for payments, delegated actions, or fine-grained consent

RFC 9396

APPENDIX

FAPI 2.0

- **Means:** Financial-grade API Security Profile 2.0
- **Benefit:** Turns best practices into enforceable high-assurance policy
- **How:** Requires controls such as PAR, code flow, S256 PKCE, strong client authentication, and sender constraint
- **Use when:** API access is regulated, high-value, or ecosystem-governed

APPENDIX

JWT Access Token Profile

- **Means:** Standard profile for JWT access tokens
- **Benefit:** Improves interoperability for self-contained access tokens
- **How:** Defines common claims and validation expectations for resource servers
- **Use when:** APIs validate JWT access tokens locally

RFC 9068

Token Introspection

- **Means:** OAuth 2.0 Token Introspection
- **Benefit:** APIs can ask the authorization server about token state
- **How:** Resource server posts a token and receives metadata such as `active`, subject, scope, and audience
- **Use when:** Opaque tokens, central revocation, or live status checks are required

RFC 7662

Token Revocation

- **Means:** OAuth 2.0 Token Revocation
- **Benefit:** Clients can explicitly invalidate tokens
- **How:** Client calls the revocation endpoint with a token and optional token type hint
- **Use when:** Logout, compromise, or client cleanup should invalidate refresh or access tokens

RFC 7009

Token Exchange

- **Means:** OAuth 2.0 Token Exchange
- **Benefit:** Avoids forwarding broad or wrong-audience tokens downstream
- **How:** Client exchanges a subject token for another token with a different audience, resource, or scope
- **Use when:** APIs call APIs on behalf of users or workloads

RFC 8693

Resource Indicators

- **Means:** OAuth 2.0 Resource Indicators
- **Benefit:** Helps clients request tokens for the right API
- **How:** Authorization and token requests can name a target `resource`
- **Use when:** One authorization server issues tokens for multiple APIs

RFC 8707

Protected Resource Metadata

- **Means:** Metadata for OAuth protected resources
- **Benefit:** Helps clients discover how an API expects tokens
- **How:** Resource server publishes metadata such as authorization servers, scopes, and presentation methods
- **Use when:** APIs and clients participate in dynamic or multi-resource ecosystems

RFC 9728

OpenID Federation

- **Means:** Federation framework for OpenID ecosystems
- **Benefit:** Scales trust establishment across organizations
- **How:** Entities publish signed statements that describe metadata, trust chains, and policies
- **Use when:** Trust is decentralized and cannot be managed as static client registrations

AuthZEN

- **Means:** OpenID Authorization API
- **Benefit:** Standardizes externalized authorization decision requests
- **How:** Applications ask a policy decision point whether a subject can perform an action on a resource
- **Use when:** Authorization decisions need central policy, auditability, or cross-service consistency

APPENDIX

Passkeys And WebAuthn

- **Means:** Phishing-resistant public-key user authentication
- **Benefit:** Removes shared passwords from the login ceremony
- **How:** Authenticator signs a challenge scoped to the relying-party origin
- **Use when:** User authentication should resist credential phishing and password reuse

WebAuthn · FIDO Passkeys

RFC References / 1

RFC 6749

RFC 7515

RFC 7516

RFC 7519

RFC 7636

RFC 7638

RFC 6750

RFC 7009

RFC 7662

RFC References / 2

RFC 8693

RFC 8705

RFC 8707

RFC 9068

RFC 9101

RFC 9126

RFC 9207

RFC 9396

FURTHER READING

RFC References / 3

RFC 9449

RFC 9470 — API step-up

RFC 9700

RFC 9728

FURTHER READING

References / 1

OAuth 2.1 draft

Browser-based apps (RFC 10017)

OpenID Connect Core

FAPI 2.0 Security Profile

JARM

OpenID Federation 1.0

OpenID Federation 1.1

OpenID Authorization API 1.0

FURTHER READING

References

W3C WebAuthn Level 3

FIDO Passkeys

ProAuth BFF docs

ProAuth OIDC client docs